

Project Documentation

Full Stack Development – Project Documentation Format

1. Introduction

Project Title	Implement Client Script & UI Policy (Incident)
Platform	ServiceNow (low-code platform)
Date	03-10-2026
Team ID	SWTID-2026-8713

Team Members

S.No	Team Member	Role	Responsibility
1	Vishal	Requirements & Configuration	Identify Incident fields and conditions; rule maintenance
2	Thabeshwin	UI Policy Developer	UI Policies for mandatory and read-only fields
3	Supriya	UI Policy & Client Script Developer	UI Policy visibility; onLoad, onChange and onSubmit Client Scripts
4	Sakthivel S	Tester & Documentation	Testing, validation, project documentation and report

2. Project Overview

Purpose

This project demonstrates how to use Client Scripts and UI Policies on the ServiceNow Incident form to control fields dynamically. Fields become mandatory, read-only or visible when defined conditions are met, and the form reacts immediately to what the user does, without being submitted. The goal is more complete and consistent Incident data and a shorter, clearer form for service desk agents.

Features

- UI Policies that make Close code and Close notes mandatory when State is Resolved or Closed, and Short description mandatory for every new incident
- UI Policies that show or hide fields (for example On hold reason) and make fields read-only on closed incidents
- onLoad Client Script that sets the initial form state and shows guidance
- onChange Client Scripts on Category, State, Impact and Urgency that update dependent fields
- onSubmit Client Script that validates the data and blocks an invalid save with a clear message
- Rules that can be activated, deactivated and updated by an administrator
- Verification by changing fields on the form and checking the browser console

3. Architecture

The project is built entirely on the ServiceNow platform, so the generic frontend / backend / database layers of the format map onto platform components as follows.

User (browser)	ServiceNow UI (Incident form)	Client-side engine (UI Policies + Client Scripts)	Table incident
Administrator, developer, service desk agent	Form and fields	Mandatory / read-only / visible rules; onLoad, onChange, onSubmit	Saved Incident records

Frontend

Instead of a React application, the user interface is the standard ServiceNow Incident form, opened from the All menu with incident.list or incident.do. UI Policies and Client Scripts run in the browser and change the fields as the user works.

Backend

Instead of Node.js and Express.js, the server side is the ServiceNow platform. The rules themselves are stored as records in the sys_ui_policy and sys_script_client tables and are delivered to the browser with the form. Access is still enforced on the server by roles and ACLs; client-side rules guide users but are not a security control.

Database

Instead of MongoDB, data is stored in the existing Incident table (incident). No new table is created. The fields controlled by the rules are:

Field	Label	Rule applied
short_description	Short description	Mandatory for every new incident; validated onSubmit
state	State	Trigger field for UI Policies and onChange script
category / subcategory	Category / Subcategory	onChange script clears dependent values
impact / urgency	Impact / Urgency	onChange guidance message
close_code / close_notes	Close code / Close notes	Mandatory when State is Resolved or Closed
hold_reason	On hold reason	Visible only when State is On Hold

4. Setup Instructions

Prerequisites

- A ServiceNow instance (such as a personal developer instance) with admin access
- A web browser with developer tools for console checks
- Sample Incident records covering different States and Categories

Installation

1. Log in to the ServiceNow instance with admin access.
2. Go to Incident > Create New and note the fields to control (State, Category, Subcategory, Impact, Urgency, Short description, Close code, Close notes).
3. Go to System UI > UI Policies > New. Create the UI Policies for mandatory, read-only and visible fields, set their conditions on the Incident table and add the field actions. Tick "Reverse if false".
4. Go to System Definition > Client Scripts > New. Create the onLoad, onChange (Category, State, Impact, Urgency) and onSubmit scripts for the Incident table.
5. Open an incident, change the fields and check that the form reacts without being submitted.
6. Test each rule for every State and Category and check the browser console for script errors.

5. Folder Structure

ServiceNow has no client / server folders. The equivalent organisation of this project is:

Component	Location in ServiceNow	Contents
Client (UI)	All menu > Incident > Create New / incident.list	Incident form where the rules run
UI Policies	System UI > UI Policies	Mandatory, read-only and visibility rules
Client Scripts	System Definition > Client Scripts	onLoad, onChange and onSubmit scripts
Data	Incident > All	Incident records (incident table)

Component	Location in ServiceNow	Contents

6. Running the Application

No npm start is needed because the platform is hosted. To run and test the application:

- Log in to the instance and open an Incident form from the All menu.
- Change State, Category, Impact and Urgency and watch the fields become mandatory, read-only or visible.
- Try to save with missing data to see the onSubmit message.
- Open the browser console to confirm that no script errors appear.

7. API Documentation

The project does not add custom endpoints. UI Policies and Client Scripts run only in the browser form, so they do not apply to records created through APIs. The Incident table can still be reached through the standard ServiceNow endpoints below, which are protected by roles and ACLs.

Method	Endpoint	Description
GET	/incident_list.do	Opens the Incident list (UI).
GET	/api/now/table/incident	Table API read of Incident records.
POST	/api/now/table/incident	Create an incident. Client-side rules are not applied.
PUT / PATCH	/api/now/table/incident/{sys_id}	Update an incident. Server-side rules and ACLs apply.

8. Authentication

Users sign in to ServiceNow with their own credentials, and the platform maintains the session. Authorization is role-based and enforced on the server. UI Policies and Client Scripts do not change who may read or write a record; they only guide data entry on the form. The scripts use g_form to control fields.

Example – onSubmit Client Script:

```
function onSubmit() {
  var state = g_form.getValue('state');
  var shortDesc = g_form.getValue('short_description').trim();
  if (shortDesc === '') {
    g_form.showFieldMsg('short_description', 'Short description is required.', 'error');
    return false;          // stop the save
  }
  if ((state == '6' || state == '7') && g_form.getValue('close_notes').trim() === '') {
    g_form.showFieldMsg('close_notes', 'Close notes are required to resolve or close.', 'error');
    return false;          // stop the save
  }
  return true;             // data is valid, allow the save
}
```

9. User Interface

The interface is the standard ServiceNow Incident form. What the user sees changes with the State:

State	Close code / Close notes	On hold reason	Editable fields	Message shown
New / In Progress	Optional	Hidden	Yes	Guidance on load

Test ID	Scenario	Expected result	Status
PT-01	Form Load Time	Form loads in under 3 seconds.	Pending
PT-02	Field Change Response	Fields react immediately (under 1 second) without reloading the form.	Pending
PT-03	Many Rules on One Form	No slowdown, no conflicting rules and the same values always give the same behavior.	Pending

11. Screenshots or Demo

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name: Auto set urgency for high impact
Table: Incident [Incident]
UI Type: Desktop
Type: onChange
Field name: Impact

Application: Global
Active: ☒
Inherited: ☐
Global: ☒

Description:
Messages:

Script

```

1 function onChange(control, oldValue, newValue, isLoading) {
4   }
5
6   if (newValue == '1') {
7     g_form.setValue('urgency', '1');
8     g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy: High Impact Control
Table: Incident [Incident]
* Field name: Urgency

Application: Global
Mandatory: ☒
Visible: ☒
Read only: ☒
Clear the field value: ☐

Demo link: <https://github.com/<username>/<repository>> (replace with your project link)

12. Known Issues

- UI Policies and Client Scripts run only in the browser form; they do not apply to records created or changed through APIs, imports or server scripts.
- Client-side rules are not a security control; access must still be enforced by roles and ACLs.
- A UI Policy and a Client Script on the same field can conflict, so the order in which they run must be checked.
- Scripts can break if fields or choice values are renamed.

13. Future Enhancements

- Add Data Policies or Business Rules to enforce the same rules on the server for API and import updates.
- Apply the same patterns to other forms such as Problem and Change.
- Move the rules between instances with update sets.
- Add automated tests (ATF) to verify form behavior after changes.
- Add rules for mobile and Service Portal forms.